

Digital Warranty & Asset Tracking System

1. Introduction

The Digital Warranty & Asset Tracking System is a web-based platform designed to help users manage product warranties and asset details efficiently. It allows users to store product information, track warranty periods, and receive timely reminders before expiration.

The system improves organization, reduces the risk of losing warranty benefits, and simplifies asset management through a centralized digital solution.

2. Description

The **Digital Warranty & Asset Tracking System** is a full-stack web application developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js). It provides a centralized platform for managing product warranties and tracking asset-related information in a structured and organized manner.

The system allows users to add and maintain detailed records of their purchased products, including product name, purchase date, warranty duration, and invoice details. Once the product information is entered, the system automatically calculates the warranty expiration date and stores it in the database. This eliminates the need for manual tracking and reduces the chances of missing warranty deadlines.

Users can upload invoice documents or bills, which are securely stored and easily accessible whenever needed. The platform also maintains a history of maintenance or service records for each product, helping users keep track of repairs and servicing activities.

The dashboard provides a clear overview of all registered products, showing their warranty status such as active, expired, or nearing expiration. This helps users quickly identify which products require attention.

The system includes a notification feature that alerts users before warranty expiration, ensuring they can take necessary actions such as claiming warranty services or scheduling maintenance.

The platform supports role-based access, including:

User – Can manage personal products, track warranties, and view maintenance history

Administrator – Can manage users, monitor system data, and maintain platform operations

Overall, the system improves asset organization, reduces manual effort, prevents loss of warranty benefits, and enhances efficiency in managing product-related information.

3. Problem Statement

Many individuals and organizations face difficulties managing product warranties and asset information.

Some of the major problems include:

- Loss of warranty cards or purchase bills
- Forgetting warranty expiration dates
- Difficulty tracking multiple assets
- Poor asset maintenance records
- Inefficient manual record-keeping systems

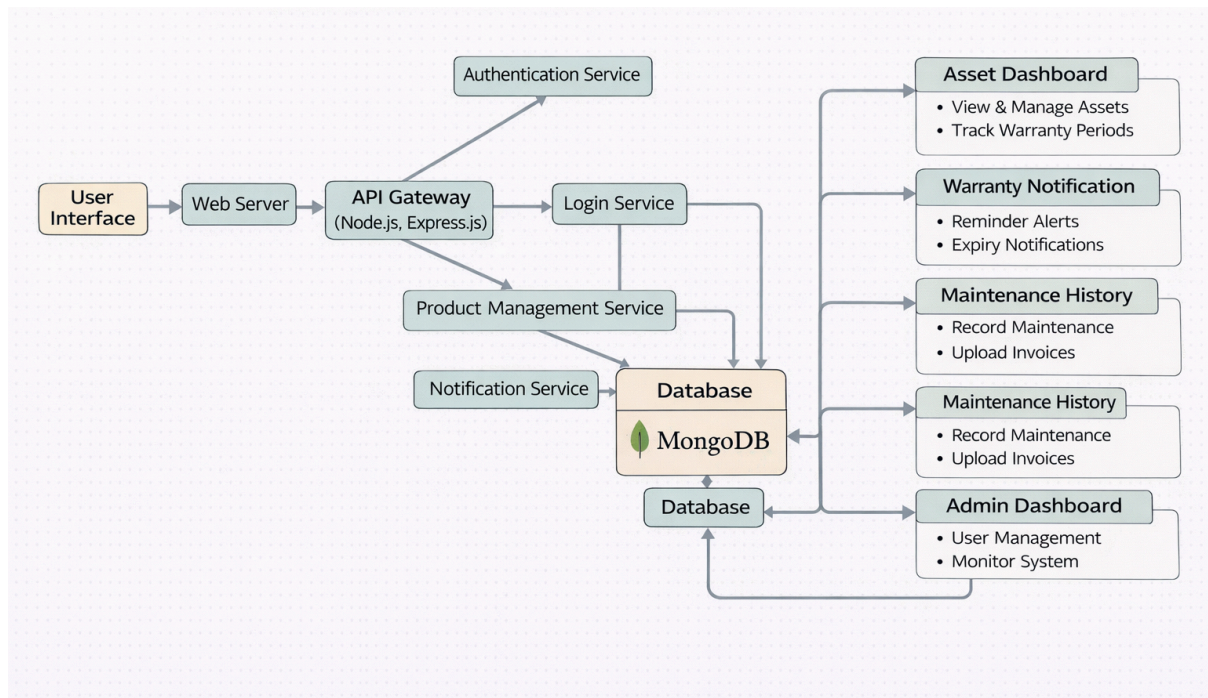
Without a structured digital system, users may lose warranty benefits or fail to maintain assets effectively.

Therefore, there is a need for a centralized digital system that allows users to track warranties, manage assets, and maintain product records securely.

4. Scenario

1. Consider a user who purchases several electronic devices such as a laptop, smartphone, and washing machine.
2. The user logs into the system and enters product details including purchase date and warranty duration.
3. The platform automatically calculates the warranty expiration date and stores it in the database.
4. The dashboard displays all products along with their warranty status.
5. Before the warranty expires, the system sends a reminder notification to the user.
6. This ensures that the user can claim warranty services before the expiration date.

5. Architecture



The **Digital Warranty & Asset Tracking System** follows a **full-stack web application architecture** built using the MERN stack.

Frontend Layer

- Built using **React.js**
- Provides dashboards for managing product details and warranty information.
- Users can:
 - Add product details
 - View warranty status
 - Upload invoices
 - Track asset information

Backend Layer

- Built using **Node.js and Express.js**
- Handles API requests and business logic.

Responsible for:

- Warranty calculations
- Asset management
- Notification services
- User authentication

Database Layer

- Uses **MongoDB**

Stores:

- User information
- Product details
- Warranty records
- Maintenance history
- Invoice files

Authentication Layer

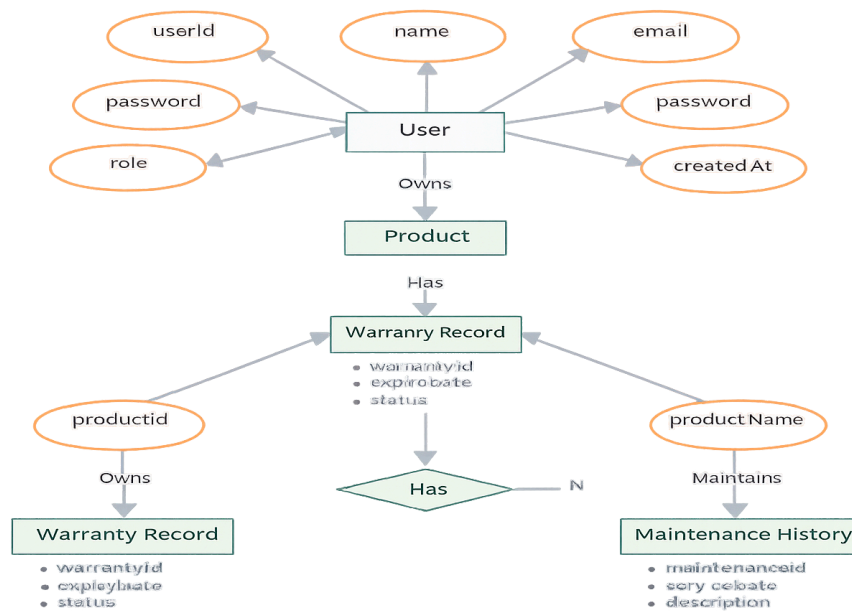
- Uses **JWT Authentication**
- Ensures secure login and role-based access control.

Notification Layer

- Handles warranty reminder notifications and alerts.

6. ER Diagram (Entity Relationship)

ER Diagram, Digital Warranty & Asset Tracking System



The Digital Warranty & Asset Tracking System uses an **Entity-Relationship (ER) model** to represent how users, products, warranties, and maintenance records are structured and interconnected.

The main entities in the system are:

1. User Entity

Stores information about users of the system.

Attributes:

- userId
- name
- email
- password
- role
- createdAt

A user can manage multiple products and assets.

2. Product Entity

Represents assets or products added by the user.

Attributes:

- productId
- userId
- productName

- purchaseDate
- warrantyPeriod
- invoiceFile
- createdAt

Each product belongs to one user.

3. Warranty Record Entity

Stores warranty details of products.

Attributes:

- warrantyId
- productId
- expiryDate
- status
- createdAt

Each product has one warranty record.

4. Maintenance History Entity

Tracks maintenance and service details of products.

Attributes:

- maintenancelId
- productId
- serviceDate
- description
- createdAt

Each product can have multiple maintenance records.

Relationships

- One **User** can have multiple **Products**
- One **Product** belongs to one **User**
- One **Product** has one **Warranty Record**
- One **Product** can have multiple **Maintenance Records**

7. Project Flow (System Flow)

Step 1: User Registration

Users create an account on the platform.

Step 2: Authentication

Users log in securely using JWT authentication.

Step 3: Product Entry

Users add product details including purchase date and warranty period.

Step 4: Warranty Calculation

The system calculates warranty expiration automatically.

Step 5: Data Storage

Product and warranty details are stored in MongoDB.

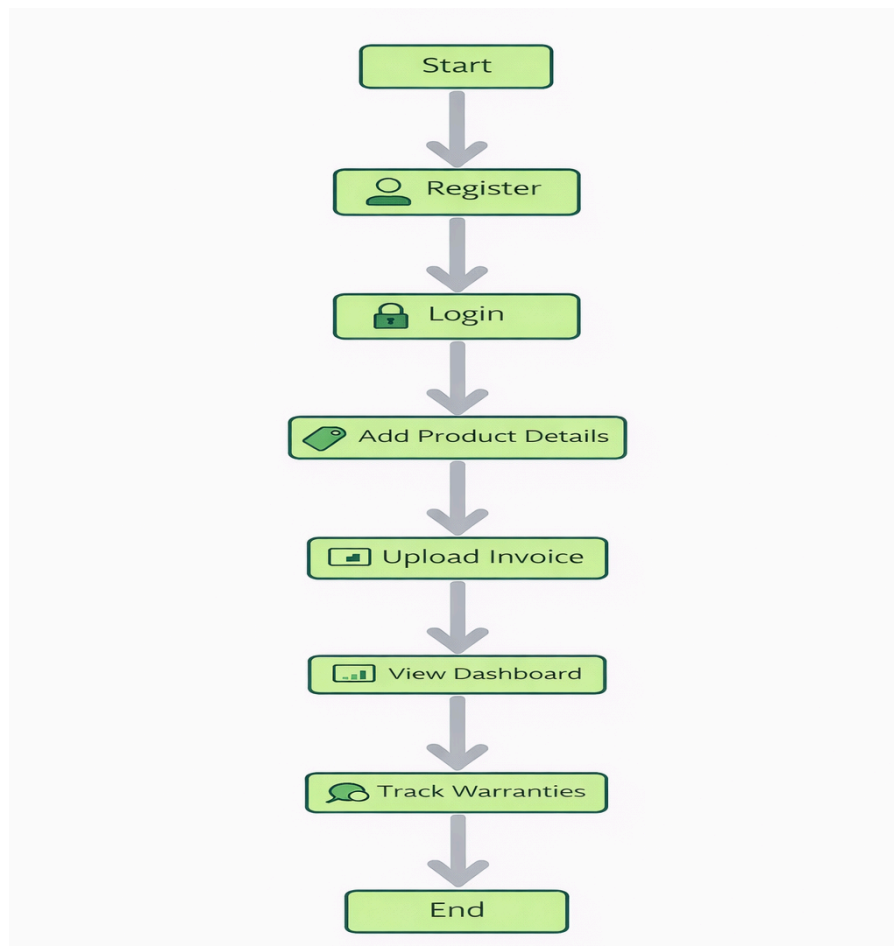
Step 6: Dashboard Monitoring

Users view all assets and warranty status through the dashboard.

Step 7: Notifications

Users receive reminders before warranty expiration.

8. User Flow



9. Pre-Requisites

To build this project, knowledge of the following technologies is required.

Frontend

- React Components
- React Hooks
- React Router
- Axios for API calls
- CSS / TailwindCSS

Backend

- Node.js fundamentals
- Express.js routing
- REST API development
- Middleware
- JWT Authentication

Database

- MongoDB document structure
- CRUD operations
- Mongoose ODM
- Schema design

Additional Concepts

- Asset management systems
- File upload handling
- Notification systems
- Form validation

10. Required Technologies

Frontend

- React.js

- TailwindCSS / Material UI
- Axios

Backend

- Node.js
- Express.js
- JWT Authentication
- Bcrypt

Database

- MongoDB
- Mongoose

Development Tools

- Git
- GitHub
- VS Code
- Postman / Thunder Client

11. Suggested Database Collections

Users

User
_id
name
email
password
role
createdAt

Products

Product
_id
userId
productName
purchaseDate
warrantyPeriod

invoiceFile
createdAt

WarrantyRecords

WarrantyRecord
_id
productId
expiryDate
status
createdAt

MaintenanceHistory

MaintenanceHistory
_id
productId
serviceDate
description
createdAt

12. Key Features

Authentication

User signup
User login
JWT authentication

Product Management

Add product details
Upload invoices
View product information

Warranty Tracking

Automatic warranty calculation
Warranty expiration tracking

Notifications

Warranty expiry reminders
Alerts for upcoming warranty expiration

Role-Based Access

Different permissions for:

- Users
- Administrators

13. Optional Advanced Features

- QR code asset tagging
- Mobile app integration
- Cloud invoice storage
- Automated email reminders
- Maintenance scheduling system
- IoT asset monitoring

14. Learning Outcomes

By completing this project, developers will learn:

- Full-stack MERN application development
- Asset management system design
- Warranty tracking systems
- REST API development
- Database schema design
- Dashboard development
- Scalable web application architecture